Student Privacy NER

30 天零基础执行计划（每天 3–5 小时）

页码基于 141 页主教程。Independent Human Learning Test: NOT VERIFIED。

Day 01 — Phase 00，第 9–12 页

知识点	项目目标、NER 输入/输出、PER/LOC/ORG、证据层级与禁止捷径。
创建/修改	阅读 README.md；创建学习日志 learning_log.md（不改模型代码）。
跟敲代码	手写 3 个输入/实体/脱敏输出例子，并区分模型测试、软件测试和人工演示。
运行命令	`git status --short; python --version`
预期输出	显示 Python 版本；Git 工作树状态可解释。
完成判断	能独立说出项目边界、三类实体和为什么不能调用现成 NER API。
失败检查	若概念混淆，回到第 9–10 页逐项写输入、处理、输出；若 Git 报错，确认当前目录。
预计时间	3 小时

Day 02 — Phase 01，第 13–18 页

知识点	Windows PowerShell、PATH、VS Code、项目目录、虚拟环境。
创建/修改	创建项目目录与 .venv；创建 learning/hello_python.py。
跟敲代码	跟敲 describe_environment()、main guard；不要复制后续文件。
运行命令	`py -3.11 -m venv .venv; .\.venv\Scripts\Activate.ps1; python -m learning.hello_python`
预期输出	输出 Python 可执行文件位于项目 .venv，并显示 CPU 环境信息。
完成判断	关闭并重开 PowerShell 后仍能激活 .venv 并运行 hello_python。
失败检查	ExecutionPolicy 错误按第 15 页处理；python 指向系统环境时重新选择 VS Code interpreter。
预计时间	4 小时

Day 03 — Phase 01，第 19–24 页

知识点	pip/uv、requirements、包安装、import 与依赖检查。
创建/修改	创建 requirements.txt、pyproject.toml；不创建模型文件。
跟敲代码	逐项理解依赖用途，安装后写三个 import 检查。
运行命令	`python -m pip install uv; uv pip install --python .venv\Scripts\python.exe -r requirements.txt; uv pip check --python .venv\Scripts\python.exe`
预期输出	All installed packages are compatible；torch/fastapi/pytest 可 import。
完成判断	能解释虚拟环境隔离什么，并在新终端验证解释器路径。
失败检查	下载失败检查网络与代理；Access denied 改用项目内 UV_CACHE_DIR；不要混用多个 Python。
预计时间	4–5 小时

Day 04 — Phase 02，第 25–28 页

知识点	list/dict/loop/function/class/self/inheritance/dataclass/type hint/半开区间。
创建/修改	创建 learning/python_foundations.py。

跟敲代码	跟敲 count_labels、NamedItem、Entity、as_dict、main，并改变一个实体输入。
运行命令	`python -m learning.python_foundations; python -m py_compile learning/python_foundations.py`
预期输出	输出实体字典和标签计数；py_compile 无报错。
完成判断	能在不看答案时写一个接收 list[str]、返回 dict 的函数，并解释 self。
失败检查	NameError 查变量作用域；TypeError 查参数；缩进错误从 traceback 最后一行定位。
预计时间	4 小时

Day 05 — Phase 03，第 29 页

知识点	Token、Sequence、Entity、Label、BIO 边界与非法 I 标签。
创建/修改	创建 learning/bio_demo.py。
跟敲代码	跟敲 validate_bio、bio_to_entities、main；手工设计一条多 Token ORG。
运行命令	`python -m learning.bio_demo`
预期输出	输出 PER/ORG/LOC 实体；故意非法 BIO 时抛出可读 ValueError。
完成判断	给任意短句手工写等长 token/label，并正确合并实体。
失败检查	token/label 数量不同先逐个编号；I-ORG 前没有 B-ORG 时修正标签。
预计时间	3–4 小时

Day 06 — Phase 04，第 30–34 页

知识点	Python package、绝对路径、配置常量、Git snapshot。
创建/修改	创建 src/、app/、tests/、data/、models/、reports/；创建 src/__init__.py、src/constants.py、.gitignore。
跟敲代码	逐行跟敲 ROOT/DATA_DIR/MODELS_DIR 等 Path 常量并解释 __file__。
运行命令	`python -m py_compile src/constants.py; python -c "from src.constants import ROOT; print(ROOT)"; git status --short`
预期输出	ROOT 等于项目根目录；语法检查无输出。
完成判断	从项目根目录和子目录调用时都能说明路径为什么稳定。
失败检查	ModuleNotFoundError 检查是否在根目录；路径错误打印 Path.resolve()；不要硬编码用户名。
预计时间	3–4 小时

Day 07 — Phase 05，第 35–38 页

知识点	JSONL、sample schema、人工标注、随机种子、数据泄漏。
创建/修改	创建 src/preprocessing.py 的 BIO 基础函数；创建 src/generate_data.py 前半部分。
跟敲代码	跟敲 _annotate、模板、实体池、generate_split；在纸上验证一个 span。
运行命令	`python -m py_compile src/preprocessing.py src/generate_data.py`

预期输出	命令无输出且 exit code 0。
完成判断	能解释 tokens 与 labels 为什么必须等长，以及 split 间实体池为何分离。
失败检查	KeyError 检查模板 placeholder；标签错位逐 token 打印；随机结果不同检查 seed。
预计时间	4–5 小时

Day 08 — Phase 05，第 39–41 页

知识点	Malaysian 合成数据、train/validation/test 隔离、重复检查与伦理限制。
创建/修改	完成 src/generate_data.py；生成 data/processed 与 data/malaysian。
跟敲代码	跟敲 generate_malaysian_split、_write_jsonl、main 和泄漏 fail-fast。
运行命令	`python -m src.generate_data; (Get-Content data\processed\train.jsonl | Measure-Object -Line).Lines`
预期输出	生成 general 2400/300/300 与 Malaysian 160/40/60；示例命令输出 2400。
完成判断	文件数量、每行 JSON、BIO 长度和 split 不重叠均可检查。
失败检查	行数不是 2400 时检查 main 的 split size；JSONDecodeError 查看出错行；不要提前依赖 Phase 08 才创建的 src.dataset。
预计时间	4 小时

Day 09 — Phase 06，第 42–46 页

知识点	Tokenizer、字符 span、normalize、Vocabulary、PAD/UNK、casing features。
创建/修改	完成 src/preprocessing.py；创建 tests/test_preprocessing.py。
跟敲代码	逐函数跟敲 tokenize_with_spans、build_vocabulary、tokens_to_ids、casing_features、save/load_json。
运行命令	`python -m pytest tests/test_preprocessing.py -q`
预期输出	6 个 preprocessing 测试通过。
完成判断	能说明每个函数输入/输出，并手算一个未知词得到 UNK id。
失败检查	span 错位检查正则 match.start/end；KeyError 检查是否使用 vocabulary.get；shape 应为 (L,4)。
预计时间	4–5 小时

Day 10 — Phase 07，第 47–52 页

知识点	Tensor、dtype、shape、Token ID、padding、mask。
创建/修改	创建 learning/tensor_demo.py。
跟敲代码	跟敲 LongTensor/FloatTensor/BoolTensor 的构造和 shape 断言。
运行命令	`python -m learning.tensor_demo`
预期输出	输出 input_ids/labels 为 int64、features 为 float32、mask 为 bool 及对应 shape。

完成判断	能解释为什么 Embedding 输入不能是 float，以及 (L,4) 中两个维度含义。
失败检查	dtype 错误打印 tensor.dtype；shape 错误逐层打印；空序列先查 tokenization。
预计时间	4 小时

Day 11 — Phase 08，第 53–55 页

知识点	Dataset、__len__、__getitem__、DataLoader、collate_fn、batch padding。
创建/修改	创建 src/dataset.py 与 learning/dataloader_demo.py。
跟敲代码	跟敲 load_jsonl、NERDataset 三个方法、collate_batch；观察两个不同长度样本。
运行命令	`python -m learning.dataloader_demo`
预期输出	输出 padded batch、mask 和统一的 (batch, max_length) shape。
完成判断	能从 row 追踪到 batch 中 input_ids/features/labels/mask。
失败检查	stack 报错通常是未 padding；padding id 混乱检查 constants；DataLoader 空检查数据路径。
预计时间	4–5 小时

Day 12 — Phase 09，第 56–61 页

知识点	Neural Network、Module、Layer、Parameter、Embedding。
创建/修改	按顺序创建 tutorial/model_versions/model_v1.py 与 model_v2.py。
跟敲代码	只跟敲 V1/V2；比较新增的 __init__ 参数、super() 和 embedding 输出。
运行命令	`python -m pytest tests/test_model_versions.py -q -k "version_1 or version_2"`
预期输出	V1/V2 对应测试通过；Embedding shape 为 (batch,length,embedding_dim)。
完成判断	能说明 embedding table 哪些值会训练，以及 padding_idx 的作用。
失败检查	AttributeError 检查 super().__init__；索引越界检查 vocab_size；dtype 必须 long。
预计时间	4–5 小时

Day 13 — Phase 09，第 61–65 页

知识点	RNN 局限、LSTM gates、hidden state、双向上下文。
创建/修改	创建 model_v3.py 与 model_v4.py。
跟敲代码	跟敲 BiLSTM、Linear、Dropout；画出 forward/backward 两条信息流。
运行命令	`python -m pytest tests/test_model_versions.py -q -k "version_3 or version_4"`
预期输出	V3/V4 测试通过；BiLSTM 最后一维为 2×hidden_dim。
完成判断	能比较 LSTM 与 BiLSTM，并解释为什么姓名识别需要左右文。
失败检查	维度不匹配检查 bidirectional=True 和 Linear 输入；Dropout 只在 train 模式随机。
预计时间	4–5 小时

Day 14 — Phase 09，第 65–70 页

知识点	forward 数据流、特征拼接、logits 与完整 shape tracking。
创建/修改	创建 model_v5.py、model_v6.py；最终创建 src/model.py。
跟敲代码	逐版本跟敲 forward；明确 V5→V6 新增 features projection 与 cat。
运行命令	`python -m pytest tests/test_model_versions.py tests/test_model.py -q`
预期输出	7 个模型相关测试通过；最终 logits shape=(batch,length,8)。
完成判断	不看代码画出 input_ids/features→embedding→BiLSTM→Linear→logits。
失败检查	matmul shape 错误按每层打印 shape；num_labels 应与标签映射一致；不要加 softmax。
预计时间	5 小时

Day 15 — Phase 10，第 71–73 页

知识点	CrossEntropyLoss、gradient、zero_grad、backward、optimizer.step、gradient clipping。
创建/修改	创建 learning/training_step_demo.py。
跟敲代码	跟敲一次完整训练 step，并在 step 前后复制 classifier 权重比较。
运行命令	`python -m learning.training_step_demo`
预期输出	输出 finite loss、logits (1,4,8)，并确认 classifier changed=True。
完成判断	能准确区分 backward 计算梯度与 step 更新参数。
失败检查	loss nan 检查标签范围；参数不变检查是否漏 step；梯度累积检查是否漏 zero_grad。
预计时间	4–5 小时

Day 16 — Phase 11，第 74–78 页

知识点	Epoch、train/eval、no_grad、validation、checkpoint、随机种子。
创建/修改	创建 src/metrics.py 与 src/train.py；创建 models/、reports/ 输出目录。
跟敲代码	先跟敲 set_seed 和 run_epoch，再跟敲 main；标出 optimizer=None 的 validation 分支。
运行命令	`python -m py_compile src/train.py src/metrics.py; python -m src.train`
预期输出	CPU 完成 16 epochs；生成 models/best_model.pt、vocabulary.json、labels.json、config.json、training_history.json。
完成判断	能打开 history 说明 train/validation loss，并解释为何按 validation 保存。
失败检查	训练慢确认 CPU 正常；找不到数据先运行 generate_data；OOM 降低 batch 而不改模型定义。
预计时间	5 小时（可让训练继续运行）

Day 17 — Phase 12，第 79–81 页

知识点	map_location、eval/no_grad、argmax、BIO decoding、字符 offset。

创建/修改	创建 src/predict.py。
跟敲代码	跟敲 Predictor.__post_init__、predict_tokens、predict、extract_entities。
运行命令	`python -c "from src.predict import Predictor; print(Predictor().predict('Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru.'))"`
预期输出	返回 text/tokens/tags/entities；实体包含 text/type/start/end。
完成判断	能追踪 raw text→tokens→ids→logits→tags→entities。
失败检查	checkpoint key 错误核对保存结构；CPU load 使用 map_location；标点被吞检查标点 tag 归 O。
预计时间	4–5 小时

Day 18 — Phase 13，第 82–85 页

知识点	半开字符区间、从右向左替换、保留空格/标点/换行。
创建/修改	创建 src/redact.py 与 tests/test_redaction.py。
跟敲代码	跟敲 redact_tokens 和 redact_text；先在纸上演算两个 span 的正序/逆序差异。
运行命令	`python -m pytest tests/test_redaction.py -q`
预期输出	3 个脱敏测试通过；原文非实体内容保持不变。
完成判断	能解释为什么字符替换必须从最大 start 向左进行。
失败检查	位置漂移检查排序 reverse=True；空格变化使用 redact_text 而不是重新 join token。
预计时间	3–4 小时

Day 19 — Phase 14，第 86–93 页

知识点	本地数据合并、domain adaptation、合成数据限制、独立 Malaysian test。
创建/修改	检查并必要时修改 src/train.py 的 train/validation 合并；不改 test。
跟敲代码	跟敲合并逻辑与 dataset count 记录；检查 checkpoint 中 dataset_counts。
运行命令	`python -m pytest tests/test_preprocessing.py -q -k "disjoint or combined"; python -m src.train`
预期输出	泄漏测试通过；checkpoint 记录 general/Malaysian train 与 validation 数量。
完成判断	能证明 Malaysian test 60 条未进入训练，并说明合成数据不等于真实学生数据。
失败检查	指标异常高先查模板与实体池泄漏；不要将 test concatenate 到 train。
预计时间	4–5 小时

Day 20 — Phase 15，第 94–97 页

知识点	Entity-level precision/recall/F1、per-class、PERSON recall、token accuracy 陷阱。
创建/修改	创建 src/evaluate.py；检查 reports/test_metrics.json 与 malaysian_test_metrics.json。
跟敲代码	跟敲 bio_to_spans、entity_metrics 与 evaluate.main；手算 TP/FP/FN 小例子。

运行命令	`python -m src.evaluate`
预期输出	输出 general 与 Malaysian 的 PER/LOC/ORG/overall 指标并写入 reports。
完成判断	能用 TP/FP/FN 解释三项指标，并说明 PERSON recall 为什么优先。
失败检查	F1 为 0 检查实体边界是否 exact match；结果过好检查合成模板和泄漏；不要只报 accuracy。
预计时间	4–5 小时

Day 21 — Phase 10–15，第 71–97 页复习

知识点	训练—预测—评估完整链路与失败案例。
创建/修改	不新增正式文件；在 learning_log.md 记录一次失败案例。
跟敲代码	独立画数据流；修改一个非关键 demo 输入并预测结果，再运行评估。
运行命令	`python -m learning.training_step_demo; python -m src.evaluate; python -m pytest tests/test_acceptance.py -q`
预期输出	training step、两套指标与 5 个 acceptance tests 均通过。
完成判断	能在 5 分钟内口述训练、checkpoint、预测和评估的调用关系。
失败检查	某一层失败就单独运行对应测试；不要通过删断言或改 test 数据制造通过。
预计时间	4 小时

Day 22 — Phase 16，第 98–100 页

知识点	HTTP、JSON、Pydantic、status code、FastAPI dependency/cache。
创建/修改	创建 tutorial/app_versions/app_v1_text_api.py 与 tests/test_text_api.py。
跟敲代码	跟敲 TextRequest、validator、get_predictor、analyze、/api/predict、/api/redact。
运行命令	`python -m pytest tests/test_text_api.py -q; uvicorn tutorial.app_versions.app_v1_text_api:app --port 8000`
预期输出	3 个 API 测试通过；POST 返回模型实体和 redacted_text。
完成判断	能解释前端/客户端请求如何到 Predictor，再如何形成 JSON。
失败检查	422 检查 request body；500 检查 checkpoint；端口占用换 8001 并同步 URL。
预计时间	4–5 小时

Day 23 — Phase 17，第 101–103 页

知识点	multipart、MIME/extension、bytes、UTF-8、PDF text layer、5 MB 限制、OCR 边界。
创建/修改	创建 tutorial/app_versions/app_v2_file_api.py 与 tests/test_file_api.py。
跟敲代码	跟敲 analyze_uploaded_content；覆盖 txt/pdf/unsupported/oversize。
运行命令	`python -m pytest tests/test_file_api.py -q`
预期输出	TXT/PDF 走同一 Predictor；错误返回 400/413/415。
完成判断	能说明扫描 PDF 为什么没有 text layer，以及为什么本项目不声称 OCR。

失败检查	UnicodeDecodeError 检查 UTF-8；空 PDF 检查提取文本；文件大于限制应 413。
预计时间	4 小时

Day 24 — Phase 18，第 104–109 页

知识点	HTML form、DOM、fetch、JSON、可见错误与响应式页面。
创建/修改	创建最终 app/main.py 与 tests/test_web_ui.py。
跟敲代码	跟敲页面 HTML/CSS/JS、/、/api/redact 与上传路由；确认 JS 不做关键词匹配。
运行命令	`python -m pytest tests/test_web_ui.py tests/test_api.py -q; uvicorn app.main:app --reload`
预期输出	网页可打开；Redact Text 显示真实模型输出；空输入显示用户可见错误。
完成判断	用电脑和手机宽度测试输入、Clear、连续提交和换行。
失败检查	浏览器失败先看 Network status；CORS 非本例问题；模型错误看后端 traceback。
预计时间	5 小时

Day 25 — Phase 19，第 110–120 页

知识点	Unit/integration/E2E、pytest fixture/parametrize、traceback、Windows 调试顺序。
创建/修改	完成 tests/ 下所有验收文件；不修改产品逻辑来迎合错误测试。
跟敲代码	逐类阅读并跟敲至少一个 preprocessing/model/API/file/E2E test。
运行命令	`python -m pytest -q`
预期输出	56 passed。
完成判断	能把失败缩小到单文件、单测试，并解释 assertion 的预期与实际值。
失败检查	先读 traceback 最后一行；检查 cwd/interpreter/data/checkpoint；修复后先重跑单测再全套。
预计时间	4–5 小时

Day 26 — Phase 20，第 121–131 页

知识点	README、reproducibility、requirement traceability、artifact hash、GitHub 边界。
创建/修改	创建/更新 README.md、MODEL_ACCEPTANCE_REPORT.md、PRESENTATION_MATERIALS.md。
跟敲代码	把真实命令、数据规模、指标、限制和测试数写入文档；不得复制虚假结果。
运行命令	`python scripts/validate_tutorial.py; python scripts/verify_tutorial.py; git diff --check -- . ':(exclude)output/pdf/*.pdf'`
预期输出	22 Phase validation、20 path synchronization 通过；源码无 whitespace error。
完成判断	文档中的命令、测试数、指标、路径与当前仓库一致。
失败检查	旧数字用 rg 搜索；PDF 二进制从 diff --check 排除；公开 URL 未创建时不得虚构。
预计时间	4 小时

Day 27 — Phase 21，第 132–138 页

知识点	clean-room reproduction、phase checkpoint、静态同步与证据边界。
创建/修改	运行 scripts/student_reproduction_audit.py；不把最终项目整包当作教学复现。
跟敲代码	阅读脚本如何按 PHASE_FILES 累积文件；记录每个失败 Phase。
运行命令	`python scripts/student_reproduction_audit.py`
预期输出	reports/student_phase_reproduction.json 显示 22/22 PASS。
完成判断	能说明工程模拟复现不等于真人理解测试。
失败检查	耗时较长属正常；失败时查看对应 phase command/stdout/stderr，不直接跳过。
预计时间	4–5 小时

Day 28 — Phase 15、19、21，第 94–97、110–120、139–141 页

知识点	限制、失败案例、风险沟通、反馈记录。
创建/修改	创建 STUDENT_LEARNING_FEEDBACK.md；不改模型。
跟敲代码	记录耗时、难点、错误原文、缺失步骤、外部资料与是否独立完成。
运行命令	`python -m src.evaluate; python -m pytest -q`
预期输出	指标文件存在且 56 tests 通过；反馈表至少记录一个真实困难。
完成判断	能明确说模型不覆盖 email/phone/student ID，也不适合无人工复核的高风险部署。
失败检查	不要删掉失败记录；外部搜索必须记录链接和原因；指标异常先查数据。
预计时间	3–4 小时

Day 29 — Phase 20–21，第 121–141 页；INTERVIEW_PREPARATION.md

知识点	5–10 分钟英文演示、架构/数据/训练/评估/API 口述。
创建/修改	不新增产品代码；准备 demo_notes.md。
跟敲代码	按面试材料用自己的话回答 15 题；录制一次计时演示。
运行命令	`uvicorn app.main:app --port 8000; python -m src.evaluate`
预期输出	浏览器 demo 成功；演示控制在 5–10 分钟；回答引用真实指标与限制。
完成判断	不看稿也能解释 Embedding→BiLSTM→Linear、loss/backprop、F1 和 redaction。
失败检查	现场模型失败使用已保存截图/JSON 但诚实说明；回答过长压缩为问题—选择—证据—限制。
预计时间	4 小时

Day 30 — 全部 Phase，第 9–141 页

知识点	最终验收、Git 状态、可复现命令、展示顺序。

创建/修改	只修复验收发现的真实问题；不临时改架构。
跟敲代码	从 README 开始执行安装后关键命令；完成文本、TXT、PDF、Web 三种 demo。
运行命令	`python -m pytest -q; python -m src.evaluate; python scripts/validate_tutorial.py; git status --short`
预期输出	56 passed；两套评估报告；22 Phase validation；工作树状态可解释。
完成判断	全部客观标准通过、限制能说明、演示成功后才结束。
失败检查	失败按 preprocessing→model→checkpoint→API→browser 顺序隔离；不要在展示当天重新训练覆盖最佳权重。
预计时间	4–5 小时

Student Learning Feedback Form

每一天记录真实耗时、无法理解的概念、无法运行的代码、完整错误信息、缺失步骤、外部资料/帮助，以及是否独立完成。只有真实学生完成记录后，Human Learning Test 才能从 NOT VERIFIED 重新评估。

Day	Hours	Unclear concept / exact error	External help	Independent?
1				YES / NO
2				YES / NO
3				YES / NO
4				YES / NO
5				YES / NO
6				YES / NO
7				YES / NO
8				YES / NO
9				YES / NO
10				YES / NO
11				YES / NO
12				YES / NO
13				YES / NO
14				YES / NO
15				YES / NO
16				YES / NO
17				YES / NO
18				YES / NO
19				YES / NO
20				YES / NO
21				YES / NO
22				YES / NO
23				YES / NO
24				YES / NO
25				YES / NO
26				YES / NO

Day	Hours	Unclear concept / exact error	External help	Independent?
27				YES / NO
28				YES / NO
29				YES / NO
30				YES / NO